Western Engineering

Electrical and Computer Engineering

AISE 4030 Reinforcement Learning

Assignment 01: Deep Q-Networks

Released on: Thursday, February 26th, 2026
Due on: Sunday, March 15th, 2026

Note: Due date includes 48 hours grace period.

Table of Contents

1. Assignment Overview 1
2. Objectives 1
3. Environment: Super Mario Bros 1
   3.1. Installation 1
      3.1.1. Step 1: Create the Conda Environment 1
      3.1.2. Step 2: Install Dependencies 2
      3.1.3. Step 3: Verify the Installation 2
      3.1.4. Step 4: Run Training 2
   3.2. Environment Specifications 3
   3.3. Required Preprocessing 3
4. Assignment Tasks 4
   4.1. Task 1: D3QN without Experience Replay (Online Learning) 4
      4.1.1. Dueling Network Architecture 4
      4.1.2. Double DQN Target Computation 4
      4.1.3. Online Learning (No Replay Buffer) 4
   4.2. Task 2: D3QN with Uniform Experience Replay 4
      4.2.1. Replay Buffer 4
      4.2.2. Integration with the Agent 4
   4.3. Task 3: D3QN with Prioritized Experience Replay 5
      4.3.1. Sum Tree Data Structure 5
      4.3.2. Proportional Prioritization 5
      4.3.3. Importance Sampling Correction 5
      4.3.4. Integration with the Agent 5
   4.4. Task 4: Comparative Analysis 5
      4.4.1. Required Plots 5
      4.4.2. Required Discussion 5
5. Code Structure & Naming Conventions 6
6. Documentation Requirements 6
7. Training Guidelines 7
8. AI Usage Policy 7
   8.1. Academic Integrity 7
9. Submission Details 8
10. Grading Criteria 8
11. Team Collaboration 9

1. Assignment Overview

This assignment is designed to deepen your understanding of Deep Q-Network (DQN) variants by having you implement and compare three progressively more sophisticated agents on the Super Mario Bros environment. You will build all three agents using the Double Dueling DQN (D3QN) architecture, with the key variable being how each agent handles experience:

- **D3QN without Experience Replay** (online learning from single transitions)
- **D3QN with Uniform Experience Replay** (random sampling from a replay buffer)
- **D3QN with Prioritized Experience Replay** (priority-based sampling using a Sum Tree with importance sampling correction)

By holding the network architecture and learning algorithm constant (Double Dueling DQN) and varying only the experience management strategy, you can isolate and observe the specific impact of experience replay and prioritization on learning stability, convergence speed, and final performance.

Reference Material: You will be provided with a working DQN implementation for Atari Breakout. A useful starting point for the Super Mario Bros environment setup is the PyTorch Mario RL Tutorial. You must adapt and restructure this code to follow the file organization and naming conventions specified in this assignment.

2. Objectives

Upon successful completion of this assignment, you will be able to:

- Implement the Dueling Network architecture and understand how it separates state-value and advantage streams.
- Implement Double DQN target computation to reduce Q-value overestimation.
- Understand why online learning (no replay buffer) causes instability in deep RL, even with an advanced network architecture.
- Implement a uniform Experience Replay buffer and integrate it with a DQN agent.
- Implement Prioritized Experience Replay using a Sum Tree data structure with proportional prioritization and importance sampling weight correction.
- Design controlled experiments that isolate a single variable (experience management) across otherwise identical agents.
- Analyze and compare training curves to draw conclusions about convergence behavior, reward progression, and learning stability.

3. Environment: Super Mario Bros

All three agents will be trained on the Super Mario Bros environment using the gym-super-mario-bros package. The agent controls Mario on World 1-1 with a simplified action space.

3.1. Installation

Prerequisites: Anaconda or Miniconda must be installed on your machine. If you do not have it, download it from: https://docs.anaconda.com/miniconda/install/

3.1.1. Step 1: Create the Conda Environment

Open a terminal (or Anaconda Prompt on Windows) and run:

```
conda create -n AISE4030 python=3.10 -y
```

Then activate it:

```
conda activate AISE4030
```

You should see (AISE4030) at the beginning of your terminal prompt.

3.1.2. Step 2: Install Dependencies

Navigate to your project folder and install all required packages:

```
cd path/to/your-project-folder
pip install -r requirements.txt
```

Your `requirements.txt` must include the following pinned versions:

```
torch
torchvision
gymnasium==1.2.3
gym==0.26.2
gym-super-mario-bros==7.4.0
nes-py
numpy==1.26.4
matplotlib
pyyaml
```

Important: The `numpy==1.26.4` pin is required. `NumPy 2.0+` causes an `OverflowError` in the NES emulator. The `gym==0.26.2` pin is required for compatibility with `gym-super-mario-bros`.

3.1.3. Step 3: Verify the Installation

Run the following command to confirm everything is working:

```
python -c "
from training_script import make_mario_env
env = make_mario_env('SuperMarioBros-1-1-v3', render_mode=None, seed=42)
obs, info = env.reset()
import numpy as np
obs = np.array(obs)
print('Environment created successfully!')
print('Observation shape:', obs.shape)
print('Action space:', env.action_space)
env.close()
print('Setup is complete!')
"
```

You should see output similar to:

```
Environment created successfully!
Observation shape: (4, 84, 84)
Action space: Discrete(2)
Setup is complete!
```

Note: You may see a warning that says "Gym has been unmaintained since 2022..." — this is harmless and can be safely ignored.
Include this console output in your report as confirmation that your environment is set up and running.

3.1.4. Step 4: Run Training

Once verified, you can start training with:

```
python training_script.py
```

3.2. Environment Specifications

Property	Value
Environment	SuperMarioBros-1-1-v3 (via gym-super-mario-bros). World 1-1 only, single life per episode. Uses the v3 ROM variant, which provides a downsampled (pixelated) rendering — this has no effect on training since observations are preprocessed to 84 × 84 grayscale.
Raw Observation	RGB image of size 240 × 256 × 3 (Height × Width × Channels). Each pixel value ranges from 0 to 255.
Preprocessed Observation	After applying all wrappers (grayscale conversion, resize to 84 × 84, and stacking 4 consecutive frames) the final observation shape is (4, 84, 84) representing (Frames, Height, Width), with float values normalized to `[0.0, 1.0]`.
Action Space	Discrete(2). The agent can choose between two actions at each step: **action 0** (move right) and **action 1** (move right + jump). This simplified action space is enforced by wrapping the environment with `JoypadSpace` from `nes_py`, which maps these two high-level actions to the underlying NES controller inputs. The full NES controller supports 256 possible button combinations, but constraining the agent to only rightward movement and jumping reduces the exploration space and significantly speeds up learning.
Reward	Composite reward based on three components: (1) change in x-position (velocity), (2) time penalty per step, and (3) death penalty. The total reward is clipped to the range (−15, 15). Positive reward for moving right, negative for dying or standing still.
Episode Termination	The episode ends when Mario dies, reaches the flag at the end of the level, or the in-game timer reaches zero.

3.3. Required Preprocessing

Your environment.py must apply the following wrappers in order:

1. **SkipFrame:** The agent chooses an action, and that same action is repeated for 4 consecutive environment frames. The rewards from all 4 frames are summed and returned as a single reward. This reduces the decision frequency from ~60 decisions/second to ~15, speeding up training without losing meaningful control.
2. **GrayScaleObservation:** Convert RGB frames to grayscale. Mario's actions do not depend on color information, so this reduces the observation from 3 channels to 1 without losing useful information.
3. **ResizeObservation:** Downsample frames to 84 × 84 pixels to match the standard DQN input size and reduce computational cost.
4. **FrameStack:** Stack 4 consecutive preprocessed frames into a single observation of shape (4, 84, 84). This gives the agent temporal context, allowing it to perceive motion and direction from the stacked frames.

Refer to the PyTorch Mario tutorial for details on the wrapper implementation. Your `environment.py` must encapsulate all of this setup and return the environment, observation shape, and action size.

4. Assignment Tasks

4.1. Task 1: D3QN without Experience Replay (Online Learning)

Implement a Double Dueling DQN agent that learns online from single transitions, without any replay buffer. This serves as your baseline and demonstrates why experience replay is critical, even when the network architecture itself is sophisticated.

4.1.1. Dueling Network Architecture

Your d3qn_network.py must implement the Dueling Network architecture. After the shared convolutional feature extractor, the network splits into two separate streams:

- **Value Stream V(s):** A fully connected layer that outputs a single scalar estimating the value of being in state s.
- **Advantage Stream A(s, a):** A fully connected layer that outputs one value per action, estimating the relative advantage of each action.

These streams are recombined using the formula:

$$\boldsymbol{Q(s,a) = V(s) + A(s,a) - \mathrm{mean}\big(A(s,\cdot)\big)}$$

Subtracting the mean advantage ensures identifiability — the network cannot simply shift values between **V** and **A** without changing **Q**.

4.1.2. Double DQN Target Computation

Your agent must use Double DQN logic for computing TD targets. Instead of using the target network to both select and evaluate the best next action (which causes overestimation), the process is split:

1. **Action Selection:** Use the policy network to select the best action for the next state: $a' = \mathrm{argmax}\, Q_{\mathrm{policy}}(s,.)$
2. **Action Evaluation:** Use the target network to evaluate that action: $Q_{\mathrm{target}}(s', a')$
3. **TD Target:** $\mathrm{target} = \boldsymbol{r} + \boldsymbol{\gamma} \cdot Q_{\mathrm{target}}(s', a') \cdot (1 - \mathrm{done})$

4.1.3. Online Learning (No Replay Buffer)

In this task, the agent performs a gradient update after every single environment step, using only the current transition $(s, a, r, s', \mathrm{done})$. There is no replay buffer. The agent must still use a target network with periodic syncing.

Expected behavior: This agent should exhibit unstable learning. Reward may increase briefly, but is likely to regress due to catastrophic forgetting caused by correlated sequential samples. This is the expected outcome and motivates Task 2.

4.2. Task 2: D3QN with Uniform Experience Replay

Add a standard uniform replay buffer to your D3QN agent. The network architecture and Double DQN target computation remain identical to Task 1 — the only change is how the agent stores and samples experience.

4.2.1. Replay Buffer

Implement a replay buffer in `replay_buffer.py` that stores transitions $(s, a, r, s', \mathrm{done})$ up to a fixed capacity. When the buffer is full, the oldest transitions are overwritten. The agent samples a random mini-batch of transitions from the buffer for each learning step.

4.2.2. Integration with the Agent

Your `d3qn_er_agent.py` must:

- Store every transition in the replay buffer after each environment step.
- Begin learning only after the buffer contains a minimum number of transitions (e.g., 1000).
- Sample a mini-batch of transitions uniformly at random from the buffer for each gradient update.
- Use the same Dueling Network and Double DQN target computation as Task 1.

Expected behavior: Significantly more stable learning compared to Task 1. Uniform sampling breaks temporal correlations and allows the agent to revisit past experiences, thereby reducing catastrophic forgetting.

4.3. Task 3: D3QN with Prioritized Experience Replay

Replace the uniform replay buffer with a Prioritized Experience Replay (PER) implementation. Everything else — the Dueling Network, Double DQN target computation, and hyperparameters — remains the same as Task 2.

4.3.1. Sum Tree Data Structure

You must implement **PER** using a **Sum Tree** data structure in `per_buffer.py`. The Sum Tree is a complete binary tree where each leaf stores a transition's priority, and each parent stores the sum of its children. This enables *O(log n)* sampling proportional to priority.

Your Sum Tree implementation must support:

- Adding a new transition with a given priority.
- Updating the priority of an existing transition.
- Sampling a transition proportional to its priority value.
- Retrieving the maximum priority currently stored (for assigning initial priorities to new transitions).

4.3.2. Proportional Prioritization

Transition priorities are based on the absolute TD error plus a small constant:

$$\boldsymbol{P_i} = (|\boldsymbol{\delta_i}| + \boldsymbol{\epsilon})^{\boldsymbol{\alpha}}$$

where δ_i is the TD error, ε is a small positive constant (e.g., 1e-5) to prevent zero priority, and α controls how much prioritization is used (α = 0 is uniform, α = 1 is full prioritization). Use α = 0.6 as the default.

4.3.3. Importance Sampling Correction

Prioritized sampling introduces bias because transitions are no longer sampled uniformly. To correct this, each sampled transition is weighted by an importance sampling (IS) weight:

$$\boldsymbol{w_i} = \left(\boldsymbol{N} \cdot \boldsymbol{P}(\boldsymbol{i})\right)^{(-\boldsymbol{\beta})} / \mathbf{max}(\boldsymbol{w})$$

where N is the buffer size, P(i) is the sampling probability of transition i, and β controls the degree of correction. β is annealed linearly from an initial value (e.g., β = 0.4) to 1.0 over the course of training. The weights are normalized by dividing by the maximum weight in the batch for stability.

These IS weights must be multiplied with the loss for each transition in the mini-batch before computing the gradient.

4.3.4. Integration with the Agent

Your `d3qn_per_agent.py` must:

- Store transitions in the PER buffer with an initial priority equal to the current maximum priority in the tree.
- Sample mini-batches using proportional prioritization via the Sum Tree.
- Compute IS weights and apply them to the loss.
- Update the transition priorities in the tree after each learning step based on the new TD errors.
- Anneal β linearly from its initial value to 1.0 over training.

4.4. Task 4: Comparative Analysis

After training all three agents, produce a written analysis comparing their performance. This section is graded on the quality of your reasoning about your own experimental results. Generic explanations copied from textbooks or AI tools will not receive full marks; you must reference specific behaviors observed in your training curves.

4.4.1. Required Plots

Generate the following plots and include them in your report:

1. **Reward Curve (all three agents overlaid):** Episode reward (moving average) over training episodes for all three variants on the same plot.
2. **Loss Curve (all three agents overlaid):** Average episode loss over training episodes for all three variants on the same plot.
3. **Per-agent detailed plots:** Individual reward and loss plots for each agent (similar to the Naive DQN plots you received as reference).

4.4.2. Required Discussion

Your written analysis must address the following questions. For each, cite specific evidence from your plots:

1. **Convergence Comparison:** Which agent converged fastest? Which achieved the highest reward? Did any agent fail to converge, and if so, what do the training curves show?
2. **Stability Analysis:** Compare the stability of each agent's learning curve. Which showed the most variance? Did any agent exhibit catastrophic forgetting or reward regression?
3. **Effect of Experience Replay:** What specific difference did adding a uniform replay buffer make compared to online learning? Reference the episode or step range where the difference became apparent.
4. **Effect of Prioritization:** Did PER improve over uniform replay? If so, in what way (faster convergence, higher final reward, or both)? If not, provide a hypothesis for why.

5. Code Structure & Naming Conventions

Your project must follow the file structure below. This structure ensures consistency across all submissions and prepares you for comparing implementations.

```
Super Mario D3QN/
├── config.yaml                # All hyperparameters and settings
├── environment.py             # Mario environment setup and wrappers
├── d3qn_network.py            # Dueling Network architecture (shared by all agents)
├── d3qn_agent.py              # D3QN agent WITHOUT experience replay (Task 1)
├── d3qn_er_agent.py           # D3QN agent WITH uniform experience replay (Task 2)
├── d3qn_per_agent.py          # D3QN agent WITH prioritized experience replay (Task 3)
├── replay_buffer.py           # Uniform replay buffer
├── per_buffer.py              # PER buffer with Sum Tree + importance sampling
├── training_script.py         # Main entry point: training, evaluation, deployment
├── utils.py                   # Config loading, plotting, logging, helpers
├── README.md                  # Project overview, setup instructions, how to run
├── requirements.txt           # Python dependencies
├── d3qn_results/              # Task 1 outputs (model, history, plots)
├── d3qn_er_results/           # Task 2 outputs
└── d3qn_per_results/          # Task 3 outputs
```

The `config.yaml` must control which agent variant is trained. Your `training_script.py` should read a mode or agent type parameter from the config and instantiate the appropriate agent and buffer. A single training script should be able to run all three experiments by changing only the config file.

6. Documentation Requirements

Each class and method in your code must include a complete Python docstring that describes its purpose, parameters (with types), and return values (with types). This applies to all files, not just agent classes.
Example of required documentation style:

```
def sample(self, batch_size):
    """
    Samples a batch of transitions proportional to their priority.

    Args:
        batch_size (int): Number of transitions to sample.

    Returns:
        tuple: (states, actions, rewards, next_states, dones, indices, weights)
            - indices (np.ndarray): Buffer indices of sampled transitions
              (needed for priority updates).
            - weights (np.ndarray): Importance sampling weights for each
              sampled transition.
    """
    Pass
```

7. Training Guidelines

The following table provides recommended hyperparameters. You may adjust these based on your experimentation, but you must document any changes and justify them in your report.

Hyperparameter	Recommended Value	Notes
Total training episodes	5,000 – 10,000	Adjust based on available compute
Batch size	32	For agents with replay buffers (Tasks 2 and 3)
Replay buffer capacity	100,000	Maximum transitions stored
Learning starts	1,000	Min buffer size before learning begins
Learning rate	0.00025	Adam optimizer
Discount factor (γ)	0.9	As used in the Mario tutorial
Epsilon start / min	1.0 / 0.1	Exploration schedule
Epsilon decay	0.99999975	Multiplicative decay per step
Target network sync	Every 10,000 steps	Copy policy network weights to target
PER α	0.6	Prioritization exponent
PER β (initial)	0.4	Annealed to 1.0 over training
PER ϵ	1e-5	Small constant to prevent zero priority
Gradient clipping	1.0	Max gradient norm
Loss function	Huber Loss (SmoothL1)	Robust to large TD errors

Important: All three agents must be trained with the same hyperparameters (where applicable) to ensure a fair comparison. The only variables between experiments should be the experience management strategy.

8. AI Usage Policy

You are permitted and encouraged to use AI tools (e.g., ChatGPT, Claude, GitHub Copilot, Gemini, etc.) to assist you throughout this assignment. However, transparency and accountability are required.
You must include an AI Usage Log as an appendix to your submission. This log must contain:

- **Tool Identification:** The name and version of each AI tool used.
- **Purpose:** A brief description of how the tool was used.
- **Full Prompt Log:** A complete list of all prompts you submitted, in order. You may paraphrase very long conversations, but the substance of every prompt must be captured.
- **What You Changed:** Whether you used the AI output as-is or modified it, and what changes you made.

8.1. Academic Integrity

Using AI without documenting it in your AI Usage Log will be treated as a violation of academic integrity. You are fully responsible for understanding and being able to explain every line of code you submit. You may be asked to explain any part of your implementation during grading.

9. Submission Details

You must submit two (2) items to the assignment portal:

1. PDF Report (3–5 pages): A report containing:

- Confirmation that your environment is set up and running (include console output showing observation space, action space, and device).
- Training results: all required plots from Section 4.4.1.
- Comparative analysis addressing all questions from Section 4.4.2.
- A brief description of any hyperparameter changes you made and why.
- AI Usage Log (as described in Section 8), included as an appendix.
- Team Contribution Efforts: Each member must outline their specific contributions in short bullet points.

2. Code Archive (.zip file): A single .zip file containing:

- All code files follow the structure in Section 5.
- `config.yaml` with all hyperparameters used.
- `README.md` with setup instructions and how to run each agent.
- `requirements.txt` listing all Python dependencies.
- Results directories with saved models, training histories, and plots.

10. Grading Criteria

Criteria	Description	Points
Task 1: D3QN (No ER)	Correct Dueling Network architecture, Double DQN target computation, and online learning agent. Code is well-documented and runs without errors.	20
Task 2: D3QN + ER	Correct uniform replay buffer implementation and integration with the D3QN agent. Proper mini-batch sampling and learning-start threshold.	20
Task 3: D3QN + PER	Correct Sum Tree implementation, proportional prioritization, importance sampling weights with β annealing, and priority updates. This is the most technically demanding task.	25
Comparative Analysis	All required plots are included. Discussion is specific to the team's own results, references concrete evidence from training curves, and demonstrates understanding of why each variant behaves as observed.	20
Code Organization	File structure follows Section 5 conventions. Includes config.yaml, README.md, and requirements.txt. All classes and methods have complete docstrings.	5
AI Usage Log	Complete and honest documentation of AI tool usage, including all prompts and modifications.	5
Professionalism	Report is clear, well-written, and properly formatted. Team contributions are included.	5
Total		100

11. Team Collaboration

This is a team-based assignment using the same teams as your course project. All team members are expected to contribute. Your report must include a Team Contribution Efforts section in which each member outlines their specific contributions in short bullet points. Failure to include this section will result in a deduction under the Professionalism criterion